Farshore: Project Review

Prepared for Bibek Pathak | CMPS 4200 HCI | October 12, 2026

1. Overall assessment

Farshore is a working arcade prototype with substantial Phase 2 design coverage. Full alignment with the supplied instructor guides is not yet demonstrated by the files available. Its strongest qualities are five distinct games, visible opponent actions, fixed scoring rules, practical recovery controls, and optional pretrained-model integration. Its largest academic gaps are documented user research, a complete five-model comparison, actual peer critique, and a contribution statement that describes contributions.

The current product is a five-stop Odyssey arcade voyage. The earlier mystery and advice voyage remain playable as separate modes. The newer arcade submission package exists and describes the current direction; several older project notes have not caught up. [S3-S8]

What was verified for this review

Check	Result and practical meaning
Instructor guides	Read both original PDFs: Phase 1 has two pages; Phase 2 has three. Requirements below come from those PDFs.
Current source	Reviewed entry points, campaign rules, the five engines, input/pause controls, all three model adapters, and Vite API wiring.
Automated tests	`npm.cmd test`: 95 passed, 0 failed on October 12. These include current arcade, earlier modes, model-adapter mocks, and historical mechanics.
Production build	`npm.cmd run build` passed TypeScript and bundling. Main JavaScript: 517.27 kB minified / 169.22 kB gzip. A non-fatal chunk-size warning remains.
Submission artifacts	Inspected the arcade content, 35-page design packet, 8-slide presentation PDF, one-page spike summary, raw experiment evidence, and contribution statement.
Evidence boundaries	No fresh live-model request, browser playtest, participant study, physical-phone test, or LMS submission verification was conducted in this review. Earlier checks are identified by date.

The first test attempt was blocked by the Windows sandbox from starting child processes. The permitted rerun completed successfully; this was an execution restriction, not a game-test failure.

The most useful next steps

1. Complete the evidence and submission gaps: actual interview records, model comparison, peer critique, and an accurate contribution list.
2. Test whether newcomers understand the controls, find the rivals fair, and distinguish local game AI from model preparation.
3. Improve the model's use of relevant gameplay observations and compare its benefit against a simple rule-based strategy selector.

Assessment labels: Covered means evidence is present for that requirement; Partial means some required material is present; Gap means the inspected artifact lacks required content; Unverified means the files do not establish completion. These are review judgments, not instructor grades. No later-phase guide was supplied. [S1-S2]

2. How the whole project works

The software in plain language

React builds the screens and updates them when the player acts. TypeScript checks the types of state, inputs, and responses. HTML/CSS/SVG draw most controls and boards; canvas draws the strait and archery gallery. Vite serves the website and connects browser requests to local server handlers. The optional model runs through a separately configured inference service. [S3-S6]

Layer	Main files	Responsibility
Entry and mode selection	`src/main.tsx`	Loads Arcade by default, Advice Voyage with `?voyage`, or Mystery with `?mystery`.
Campaign interface	`src/ArcadeApp.tsx`	Title, introduction, five-stop map, briefings, preferences, model preparation, results, and homecoming.
Content and progression	`src/lib/arcade.ts`	Opponents, strategy catalog, objectives, valid saves, migration, unlocks, and assisted passages.
Game rules	`src/lib/mythic.ts`	Creates a round, applies actions, advances time, moves rivals, and decides scores and wins.
Input and rendering	`src/components/arcade-round.tsx`, `mythic-games.tsx`	Countdown, keyboard/pointer controls, pause, scoreboards, grids, potion boards, melodies, and canvas scenes.
Server and model boundary	`vite.config.ts`, `server/arcade.ts`	Receives requests, validates input, calls the optional model, and checks its response.
Assets and quality checks	`public/art/`, `public/fonts/`, `tests/`	Local artwork/fonts and automated checks for rules, saves, speech, audio, and model error paths.

The player's journey

Title -> Introduction -> Map/Briefing -> Practice or Match -> Result -> Next Shore -> Homecoming. New voyages begin at the Cyclops. A clear or confirmed assisted passage unlocks the next shore. Practice does not advance progress. Failed replays preserve earlier clears; a successful replay can upgrade an assisted passage. [S4]

During a round, the shared component reads player actions and advances the engine on animation frames. The engine copies and updates state, then the interface redraws it. A fixed seed reproduces the same random sequence, which helps test failures reliably. The game handles scores and collisions locally. [S5-S6]

Optional model preparation

Browser settings + up to two attempt summaries -> POST `/api/arcade` -> server-derived observations -> model strategy choice -> validation -> player review -> local opponent play.

The model chooses between two allowed adaptive strategies for the selected encounter. The server fixes the required `focus`, validates the returned fields, and supplies an authored description. It rejects illegal output. The model does not decide scores, shorten warnings, or run a real-time match. Cancellation and request-generation IDs prevent stale preparation results from replacing newer choices. [S6]

3. What players do in each mode

Current five-game arcade voyage

Game	Player actions	Win and loss rules
Polyphemus: Blind Grid Tactics	Move on an 8x8 grid using arrows/WASD or adjacent tiles. Sneak one tile, dash up to two, or hide on sheep. During each marked 2x2 smash, take at most one evade step.	Reach either gold exit. Three hits lose; turn limits are 32/26/22 on Easy/Normal/Hard. Sheep conceal quiet moves; dashes leave louder clues.
Circe: Transmutation Match-3	Choose a column with Left/Right, cycle the triplet's order with Up, and drop with Space. Match at least three identical ordinary tiles in a row or column. Moly clears swine/curse tiles; chains and Moly send curses.	Circe's board filling first wins. Your board filling loses. At two minutes, higher brew points wins; a tie favors Circe. Pieces also drop automatically after their delay.
Sirens: Melody Memory	Watch/listen first. During YOUR REPLY, echo notes with A/S/D/F, 1/2/3/4, or the pads. Skip rests and finish before the reply deadline.	Five correct melodies win. Wrong or missing notes increase Temptation; 100% loses. A correct melody lowers it.
Scylla and Charybdis: Paddle Defense	Move the shield with Up/Down or W/S, or drag on the sea. Align with incoming strikes. Hold Space or Row to resist the vortex.	Survive 30/40/50 seconds on Easy/Normal/Hard. Five hull hits or reaching the vortex loses.
Antinous: Twelve-Ring Archery	Adjust aim against wind, elevation, and tension. Use the trajectory preview, then release with Space or the button. Antinous takes his own turn before new wind arrives.	Higher total rings after five shots each wins. Ties favor Antinous. One adaptive strategy can partly block a ring for your next shot.

Rules and controls are implemented in the current engines and shared interface. Keyboard, pointer, and visible cues are supported; that does not establish complete assistive-technology accessibility. [S4-S5]

The three opponent options

Arcade Solo uses authored, learnable policies. **Local AI** immediately runs adaptive game heuristics. **Model-planned AI** uses a pretrained model to select a policy before that same local engine executes it. A heuristic is valid game AI, but only the third option demonstrates the arcade's pretrained-model integration. [S4-S6]

The retained earlier modes

Mystery (`?mystery`): Inspect three rooms, collect evidence and items, question witnesses, repair a seal, open a gate, and support a deduction. Stars represent escape, proof, and optional motive discovery. Optional AI matches a custom question to a currently permitted authored answer; it does not invent case facts. [S9]

Advice Voyage (`?voyage`): Advise Odysseus through fifteen authored chapters, with branching shortcuts, trust, elapsed months, and a captain's log. It starts at 120 months after Troy; the homecoming must finish below 240. Story mode uses wording rules. Optional AI interprets advice and supplies a decision, reply, and reason; code applies the authored consequences. Browser speech substitutes for the originally proposed local speech models. [S10]

4. The project's strong points

1. A complete, recognizable game loop

Farshore has a clear opening, a sequence of challenges, results, saved progress, and an ending. The five games require different skills: planning, puzzle placement, memory, coordination, and trajectory adjustment. This gives the project a stronger identity than a single repeated interaction. The breadth is a product strength; newcomer testing must establish whether switching controls is enjoyable. [S3-S5]

2. Fairness boundaries are explicit

Local code owns difficulty, collision rules, warnings, and scoring. Polyphemus locks his smash warning before the evade step. Circe has her own board and Antinous takes his own scored shot. Players can see what opponents actually do. Automated tests exercise important win/loss paths and legal boundaries. These controls support fairness, although perceived fairness still needs human evidence. [S5, S11]

3. AI failure does not block the game

The pretrained model is optional. Preparation shows elapsed time and cancellation; invalid responses cannot start a model-planned round or award progress. Players can retry, change settings, or explicitly start local play. Keeping inference outside the match also prevents a slow model call from interrupting reflex controls. [S6]

4. Several useful HCI controls already exist

Each briefing teaches an objective and controls. Short practice, retries, pause, reduced scenery motion, silent visual cues, and assistance offer different ways to learn or continue. Losing focus pauses gameplay and releases held rowing controls. Assisted completion is honestly labeled rather than counted as an ordinary skill clear. These features offer concrete behaviors to evaluate in an HCI study. [S4-S5]

5. The architecture is testable and the prototype runs locally

The rules are separated from rendering, input, and model calls. Seeded behavior makes scenarios reproducible. The current 95-test suite passes, as does the TypeScript/build check. Local assets and optional audio avoid an asset-download dependency during ordinary local play. The suite also contains legacy checks, so its total is not a count of current-arcade tests alone. [S3-S6, S11]

6. Real Phase 2 artifacts and model evidence exist

The current arcade package contains a screen hierarchy, content strategy, data flow, five persona-labeled tasks, eight edge cases, 28 wireframes, and a real six-call model spike. The wireframes include initial, loading, success, uncertain, and error states. The spike records exact provider failure bodies and measured waits instead of assuming instant responses. [S7-S8]

7. Content and privacy boundaries are documented

The story distinguishes literary episodes from invented arcade mechanics. Arcade preparation sends bounded gameplay data rather than names, voice recordings, or chat. API credentials remain server-side. In the archived experiment, inference used loopback Ollama; a future cloud setup would require a different disclosure. These are useful foundations for user trust. [S4, S6-S8]

5. Weak points and why they matter

1. Research completion is not established

The available files contain provisional sketches of Maya, Daniel, and Rosa. They do not contain six completed interviews, a synthesis, or three detailed research-backed personas. The peer-critique file is an unfilled worksheet. This limits the project's user-centered justification and leaves formal requirements unverified. Evidence may exist elsewhere; attach actual records if it does. [S1, S7]

2. The submission has a concrete contribution gap

Both the current contribution Markdown and its PDF contain only "Bibek Pathak." Phase 2 requires a short list of each member's contributions, agreed by the members. A name alone does not supply that content. Describe actual personal work and any collaborators accurately; follow any course policy on acknowledging AI assistance. The supplied brief expects 2-3 members, while the proposal says solo. No exception is documented here. [S1-S2, S7]

3. The model's additional benefit is not demonstrated

The arcade model picks one of two strategies, while the server constrains focus and local code performs all actions. That is a real, limited integration. A simple rule could also select between those strategies. No comparison currently shows that model selection improves enjoyment, learning, fairness, or retention enough to justify its waiting time. [S6-S8]

4. Some personalization inputs are inherited from older games

`mythicSummary()` currently saves `early=0`, `late=0`, and `laneMs=[0,0,0]`. The server still derives timing and lane habits from those fields, so current gameplay cannot supply those historical signals. Siren lane misses reach model preparation, but the new round initializes local miss counts to zero. `focus` is validated but is not used by the five engine decisions. This narrows personalization; it does not mean local opponents never adapt. [S5-S6]

5. Human difficulty and accessibility remain uncertain

The source implements many helpful controls, but no full newcomer study or physical-phone evaluation is supplied. The shield game combines simultaneous inputs, the puzzle displays two boards, and canvas games provide limited nonvisual information. The exact archery preview and continuously held rowing may reduce challenge for some players. Treat these as playtest questions, not confirmed user complaints. [S5, S11]

6. Three versions create maintenance and explanation costs

Old notes still say the arcade presentation needs creating, although the new package exists. Its design-packet cover says 12 slides; the actual presentation PDF has 8 and the notes describe 8. The original proposal emphasizes speech, trust, personality, and a clock; the default arcade now emphasizes skill challenges and optional policy planning. Explain that design change and keep each mode's claims distinct. [S3, S7, S10-S11]

7. Distribution is still a prototype arrangement

All three apps are imported eagerly into one main JavaScript chunk. Saves are per browser and origin, with no transfer/export feature; unfinished rounds restart after reload. Vite's API middleware runs in development and local preview, so copying `dist/` to static hosting alone does not deploy the model API. A shareable release needs a separate supported backend or an explicit local-only setup. [S3-S6]

6. Alignment with Phase 1

The original Phase 1 guide assigns 35% to the proposal, 35% to model research/selection, and 30% to user research/personas. Phase 1 was already presented according to the project handoff. This review assesses supplied evidence; it does not reopen that submission or predict a grade. [S1, S3]

Instructor requirement	Assessment	Evidence and remaining action
Problem statement and primary/secondary users	Partial	The concept and provisional player sketches exist. Document the current arcade problem and user groups explicitly, then connect them to actual research.
5-7 core features	Partial	The original slide lists six advice-game features. Current arcade capabilities are substantial, but write a coherent current feature list instead of mixing three editions.
AI improves UX and success metrics	Partial	The integration is observable and bounded. Its benefit remains a hypothesis; define measurable outcomes and test them.
Explore at least five pretrained models	Unverified	Slides name a proposed stack and claim testing; a complete comparative research record was not found. Model names alone are not an evaluation.
For each model: I/O, limitations/rate limits, response expectations, and cost	Partial	The Llama spike measures one bounded use case and documents limitations. Equivalent assessments for at least five candidate models are not supplied.
Select one or two models and justify their user fit and integration	Partial	One connected model is a permitted count. The arcade integration plan exists; explain why it is the best current choice and how the design changed from the proposed five-model stack.
Conduct at least six potential-user interviews and synthesize needs	Unverified	The original slide says interviews were booked. No completed records or synthesis are supplied. Preserve genuine records and separate research from automated tests.
Three detailed personas with all required fields	Partial	Maya, Daniel, and Rosa are sketches. Expand demographics/background, goals, pain points, tech comfort, attitudes to AI, and use scenarios from the research.
AI expectations, current workarounds, and transparency needs	Unverified	These are reflected in design hypotheses, but actual participant findings are not documented. Include them in the interview synthesis.
Team arrangement: 2-3 students	Unverified	Proposal identifies solo work. Document any instructor-approved exception or actual team arrangement; do not infer approval.

A clearer current proposal

Proposed framing to validate: Casual players and students need a short browser adventure that teaches unfamiliar controls and makes adaptive opponents understandable. Farshore offers five Odyssey challenges with practice, visible rules, and optional model-selected rival strategies.

Suggested six core features: five varied trials; briefings and practice; visible opponents and warnings; optional model preparation with review; local progress and honest assistance; accessible preferences and error recovery.

The brief requires research on at least five models and a final selection of one or two. It does not require deploying all five models from the original slides. Adding more models is therefore not the first compliance fix. [S1]

7. Alignment with Phase 2: structure and interaction

Phase 2 is primarily a design-and-evidence assignment. A working game helps demonstrate the design, but it cannot substitute for the required diagrams, states, spike, or submission content. The current arcade package provides substantial evidence for these areas. [S2, S7]

Requirement and rubric share	Assessment	Verified evidence and qualification
Information architecture: site map and content strategy (20%)	Covered	Design packet identifies S01-S15, the five game branches, shared overlays, and what each screen displays.
Mark each AI point with model, input, output, and measured time	Covered	A1 names Llama 3.2 3B, bounded preparation inputs, plan/focus output, and spike min/mean/max. A1 is shared by the five arcade briefings.
Data flow, storage, and third-party boundary	Covered	Diagram follows browser -> server -> loopback model -> validated result -> review/local game, with browser localStorage. Future hosting is explicitly distinguished.
3-5 primary tasks labeled with Phase 1 personas (30%)	Covered, with research limit	F1-F5 cover learning, preparation, gameplay, recovery, and completion. Persona labels are present, but the underlying personas remain provisional.
Input, processing, result, recovery, and user control flows	Covered	F2/F4 show waiting, Cancel, review, regeneration, mode/difficulty changes, and explicit fallback. A dedicated Discard is proposed; existing setting changes can reject a plan.
At least three required edge cases	Covered	EC1-EC8 include service failure/timeout/429, poor or empty output, unsupported input, late replies, inappropriate output, lost focus, saving failures, and migrated saves.
Every site-map screen and major state wireframed (30%)	Covered in supplied packet	W01-W28 cover screens and additional gameplay/overlay states. Editable SVGs and content data accompany the 35-page PDF.
All five AI states and flow/state annotations	Covered	W05 initial, W06 loading, W07 success, W08 uncertain, W09 error. W10 also covers unsupported input.
Appropriate confidence/result communication	Covered with improvement opportunity	No invented confidence percentage. Authored strategy descriptions and review controls are present. Legal strategy quality is not established by schema validation.
Structured class critique scheduled in the brief	Unverified	`peer-critique.md` is a worksheet, not evidence of feedback given/received and resulting revisions.

Two design improvements that would strengthen the argument

Make rejection explicit. Add a Discard strategy control that retains AI mode and returns to the initial preparation state. This would make the existing rejection behavior easier to discover. It is a recommendation, not a claim that the brief requires a button with that exact name.

Explain the observation used. Show a short, code-authored reason tied to actual history, such as two recent losses or repeated misses in one song lane. Avoid implying the model inferred a habit that the game does not collect. Describe inference location and the data sent before the user prepares a strategy. [S5-S7]

If all three editions are intended as one submitted product, extend the design mapping to the retained modes. The current arcade packet scopes itself to the arcade; do not claim that its one A1 state set documents every AI feature in the older editions.

8. Alignment with Phase 2: spike and submission

Instructor requirement	Assessment	What the supplied artifact actually shows
Standalone working code and at least five realistic inputs (spike: 10%)	Covered with one sample limitation	`phase2-arcade-spike.mjs` records six actual model calls through the production adapter. S6 injects legacy lane durations that current arcade summaries do not collect, so it tests a contract rather than a naturally observed current habit.
Record each response and min/average/max latency	Covered, small consistency issue	All six raw outputs and times are archived. Min 7.317 s; arithmetic mean 8.2785 s (8.279 rounded half up); max 11.613 s. Raw JSON shows 8.278, while the summary gives 8.279. Harmonize the stated rounding.
Trigger at least two failures and record exact results	Covered	E1 reaches the provider and gets 404 for a nonexistent model. E2 gets provider 400 for malformed JSON. E3 gets app 400 for unsupported difficulty before inference. Do not relabel E3 as a provider failure.
One-page summary: latency, quality, failure behavior, rate limits/cost, design impact	Covered with evidence limits	Summary exists and connects measured waiting to elapsed-time feedback and Cancel. It says no 429 threshold or sustained load was measured and no inference API fee was billed in this local run. Hardware/electricity remain unmeasured.
Ten minutes plus 3-5 minutes of Q&A (presentation: 10%)	Prepared; delivery unverified	Presentation PDF has 8 slides. Speaker notes total 600 seconds and reserve separate Q&A. Actual rehearsal, delivery, and who spoke are not established by the files.
Submit presentation slides	Artifact covered; submission unverified	PPTX and matching presentation PDF exist. No LMS receipt or instructor acceptance was checked.
Short contribution list agreed by all members	Gap in inspected artifact	Both current contribution files contain only the student's name. Add an accurate list of work and any actual contributors before using them as evidence of this requirement.

What the spike proves

The chosen model can return legal strategy objects for this small set of arcade examples, with recorded waits and failure behavior. Six legal responses do not establish superior strategy selection. Two choices differ from advisory prompt preferences, and the experiment does not measure learning, enjoyment, fairness, or general accuracy. [S8]

The two separately archived new-rival calls took 2.347 s for Circe and 8.838 s for Scylla. They are integration examples from a different run, not extra entries in the six-call mean. All of these measurements are historical October 8 evidence, not new October 12 measurements. [S11]

Presentation consistency and timing

The design-packet cover says 12 slides, while the actual deck has 8; correct that count. Older README/project notes incorrectly suggest no arcade packet exists. Keep one current index that directs readers to `docs/assignment/phase2-arcade/` and labels earlier artifacts as historical. [S3, S7]

The supplied brief schedules critique in the week of September 28 and Phase 2 deliverables/presentations in the week of October 5. The arcade artifacts are dated October 8. This review cannot determine whether they were submitted on time or whether the instructor accepted the concept change. Do not infer either from file dates. [S2]

9. How to take Farshore to the next level

Priority order

Priority	Concrete work	Why it matters and how to verify completion
1. Finish academic evidence	Attach actual six-interview records and synthesis; complete three personas, five-model assessment, real peer critique, and contribution list. Record the concept change and any team exception.	Closes the largest demonstrated compliance gaps. Every finding should trace to a genuine record; planned work stays labeled planned.
2. Evaluate the current games	Observe newcomers learning each trial, pause/recovery, model review, and assistance. Include keyboard-only and physical touch sessions.	Reveals control confusion, fair/unfair tactics, and barriers. Keep task results, observed failures, and revisions; playtests are distinct from the required interviews.
3. Fix observation quality	Version summaries with per-game data: cave noise/dash behavior, potion chains/overflow, song lane misses, shield misses/rowing, and archery wind compensation. Remove misleading unused habits.	Connects personalization claims to measured actions. Verify the summary-to-policy path using meaningful scenarios and update the API/save migration.
4. Test whether the model helps	Compare Solo, a deterministic policy selector, and model selection under comparable rules; counterbalance order. Repeat warm/cold latency checks and legal/invalid response probes.	Establishes whether inference earns its wait and complexity. If it adds little benefit, simplify its role while retaining a useful pretrained-model feature aligned with the course.
5. Polish onboarding and balance	Test one-action lessons, clearer shield warnings, discoverable Discard, and accessible feedback. Tune warning durations and difficulty from results. Consider an optional advanced archery preview.	Improves comprehension and challenge without expanding the game's scope. Keep assistive options available and compare the same tasks before/after.
6. Make sharing reliable	Load earlier modes on demand; add validated save export/import. For hosting, implement a supported backend and explicit provider disclosure rather than relying on static `dist/`.	Reduces initial loading, permits save transfer, and makes AI work outside the laptop. Verify deployed API behavior, privacy, and save round trips.

Suggested evaluation targets, not collected results

Measure	Proposed target for the next study
Learn controls	At least 80% complete the first practice task without moderator help.
Understand AI	At least 80% correctly identify Solo, local heuristics, and model preparation.
Recover from model failure	At least 90% continue through retry or explicit local play without losing completed progress.
Perceived fairness	Median rating at least 4/5, supported by participants' reasons and observed difficulties.
Model reliability and waiting	Report valid-response rate, warm/cold median and upper-tail wait, cancellations, and failures under stated conditions.

These targets are starting hypotheses to revise after research. Small classroom studies give useful design feedback, not population-wide proof. A short proof of benefit and better teaching would advance this project more than adding several new games immediately.

10. Operating the project and planning the next change

Practical operating limits

Run `npm.cmd run dev` from the project root and open the printed localhost URL. `npm.cmd run preview` serves the production build with the local API middleware. Keep the server running. A configured model service is separate from the website. This review did not change `.env`, credentials, game behavior, existing progress, or submission artifacts. [S3, S6]

Campaign records and preferences use browser localStorage. Arcade retains the `farshore-arcade-v1` key with schema 2, including migration of earlier three-trial saves. The older modes keep separate saves. There is no database, account, or cloud synchronization. A different browser, port, or domain can therefore show different progress. Storage failure keeps the in-memory game usable and displays a warning. [S4, S9-S10]

Where the next technical change belongs

For useful personalization, begin with `src/lib/mythic.ts`: collect the new metrics during play and emit them in `mythicSummary()`. Update the `AttemptSummary` type and validation in `src/lib/arcade.ts`, then update `server/arcade.ts` to derive meaningful observations. Keep migration compatible with existing saves. Update the briefing explanations in `src/ArcadeApp.tsx` so they describe exactly what the model can change. Verify that invalid output and late replies still preserve progress.

For maintenance, make `README.md` the current starting point, with a short version history linking the arcade package and clearly labeled earlier modes. Correct the packet's slide count and latency rounding. A contribution statement should describe actual work; it cannot be completed accurately from code alone.

For model selection, prepare one comparison row per candidate model. Record the source and date, task fit, accepted input and output, interface/schema support, practical limits, latency expectations or measured conditions, cost, and reason to select or reject it. Research at least five candidates, then justify the one or two retained. Use current primary documentation when doing that research. This report identifies the missing assessment; it does not invent a completed comparison.

A short explanation you can give your instructor

Farshore is a browser arcade adaptation of the Odyssey with five distinct games. React presents the screens, and typed local engines control scores, warnings, opponents, and progression. Players can choose learnable opponents or local adaptive rivals. An optional Llama preparation step selects one permitted rival strategy, which the player reviews before play. The Phase 2 package documents the screens, flows, wireframes, and measured model behavior. The remaining work is to support the design with real user research, strengthen the model comparison and contribution record, and test whether model-selected strategies improve the player experience.

11. Evidence references

Paths below are relative to the project root. Original instructor PDFs control requirements. Project notes and proposal summaries contain historical claims; current source and inspected artifacts control this review's description of current behavior.

ID	Source location and use
S1	`Farshore Context and Assisment/docs/assignment/original/CMPS_4200_HCI_Phase_1_Project_Requirements.pdf`, pp. 1-2: proposal, models, interviews, personas, team size.
S2	`Farshore Context and Assisment/docs/assignment/original/CMPS_4200_HCI_Phase_2_Project_Requirements.pdf`, pp. 1-3: rubric, flows, AI states, spike, timing, submissions.
S3	`README.md`, `PROJECT-NOTES.md`, `package.json`, `src/main.tsx`: entry points, setup, stack, retained modes, historical inconsistencies.
S4	`src/ArcadeApp.tsx`, `src/lib/arcade.ts`: current screens, controls, progression, strategies, saves, and preparation states.
S5	`src/lib/mythic.ts`, `src/components/arcade-round.tsx`, `src/components/mythic-games.tsx`: five engines, inputs, pause, summary data, scoring.
S6	`server/arcade.ts`, `vite.config.ts`: model catalog/schema, observations, validation, credentials boundary, timeout, and local API hosting.
S7	`Farshore Context and Assisment/docs/assignment/phase2-arcade/`: design packet, presentation PDF/PPTX, editable content/SVGs, notes, critique worksheet, contribution Markdown/PDF.
S8	S7 `evidence/phase2-arcade-spike-2026-10-08.json`, `evidence/quality-review.json`, `technical-spike-summary.pdf`; `scripts/phase2-arcade-spike.mjs`: measured calls, outputs, failures, limitations.
S9	`src/MysteryApp.tsx`, `src/lib/mystery.ts`, `server/mystery.ts`, mystery design notes: authored case and optional answer matching.
S10	`src/App.tsx`, `src/lib/game.ts`, `server/turn.ts`, `docs/assignment/PRESENTATION.md` under the context folder: advice voyage and original proposal differences.
S11	`tests/`, fresh October 12 test/build runs; context-folder `docs/arcade/trials-verification.md` and model evidence: current checks and separately dated earlier verification.